

CompanyMan App

Development Process

- Timeline:
 - **Weeks 1-2 (Planning):** Designed the database schema for Users, Menu, and Inventory. Defined the User Personas (Lucia, Ahmed, Anna) to guide feature development.
 - **Weeks 3-5 (MVP Construction):** Built the Node.js/Express backend and connected it to MongoDB. Developed the React Native frontend shell and implemented the secure Login system.
 - **Weeks 6-8 (Advanced Features):** Developed the Staff Scheduler core, subsequently integrating the Barcode Scanner for inventory and the Gemini-powered AI 'Daily Brief' agent.
 - **Week 9 (Refinement):** Focused on debugging complex state issues (like the infinite scroll bug) and polishing the UI with react-native-paper components.
- Tools and Technologies:
 - **Frontend:** React Native (Expo SDK 54), Expo Router.
 - **Backend:** Node.js, Express.js, MongoDB (Mongoose ODM).
 - **Security:** expo-secure-store for JWT storage, bcrypt for password hashing.
 - **AI:** Google Gemini API for predictive business analytics.

Design and Implementation Decisions

- Core System Functionality:
 - **Authentication & RBAC:** I implemented a Role-Based Access Control system. **Admins** manage users, **Managers** handle operations (Inventory/Sales), and **Employees** are restricted to POS and Shift views. This ensures data security and operational hierarchy.
 - **Internal POS (Point of Sale):** Instead of relying on external hardware, I built an internal POS system. It features a shopping cart state (OrderContext) that allows staff to build orders dynamically. The checkout process uses Atomic Transactions: when a sale is confirmed, the system simultaneously logs the revenue and deducts the precise ingredients from the inventory.
 - **Smart Inventory:** The inventory system goes beyond simple counting. I integrated a Barcode Scanner that uses the device's camera to scan physical products, instantly filtering the search list or pre-filling data for new items.
 - **Staff Scheduler:** Originally designed as a complex auto-scrolling list, I refactored this into a stable monthly view. It allows managers to assign shifts and visualises coverage for Morning, Afternoon, and Evening slots, with past dates visually "faded" to improve readability.
 - **Sales & AI Dashboard:** This module provides real-time financial health checks. It displays "Today's Revenue" and "Best Sellers." Crucially, it integrates **Google Gemini AI** to run background analysis on stock levels, generating natural-language "Daily Briefs" (e.g., "Milk is low, push the black coffee").

- Design Challenges & Solutions:

- **Navigation Architecture:** Early user testing showed that having separate tabs for every alert type was cluttered. I redesigned the navigation to feature a centralised **"Inbox"** tab, consolidating Shift Updates, Low Stock Alerts, and AI Suggestions into a single stream.
- **Ingredient-Level Costing:** Calculating true profit margins was complex. I implemented a recursive backend function that breaks down a menu item (e.g., "Cheesecake") into its raw ingredients (Milk, Butter), looks up their specific purchase price per unit (e.g., Price per Litre), and computes the exact food cost to display a real-time **Profit Margin** percentage on the dashboard.

Testing and Debugging

- Critical Bug Fixes:

- **The "401 Token" Race Condition:** On login, the dashboard would sometimes crash because it tried to fetch data before the secure token was fully saved to the device storage. I fixed this by implementing a "Loading Guard" in the useEffect hooks to ensure API calls only trigger after authentication is confirmed.
- **Barcode Scanner Race Condition:** Integrating the hardware camera presented significant challenges, specifically where the scanner would trigger multiple times for a single barcode due to the rapid firing rate of the onBarcodeScanned event. I resolved this by implementing a useRef locking mechanism. Unlike React state, which relies on the render cycle, the ref updates instantly, allowing the app to immediately ignore subsequent scan events. This prevented duplicate data processing and UI glitches, ensuring a reliable single-scan experience.

Security, Ethics, and Accessibility

- **Security:** All sensitive routes (like deleting users or modifying stock) are protected by double-layered middleware (auth + isManagerOrAdmin). Passwords are never stored in plain text.
- **Ethics:** The AI system is designed as "Human-in-the-loop." It offers suggestions for re-ordering or pricing, but it cannot execute financial transactions without human confirmation.
- **Accessibility:** The app uses high-contrast text (Dark Grey on White) and standard touch-target sizes (48px) to ensure usability in fast-paced, potentially dimly lit restaurant environments.

Reflection and Future Improvements

Reflection: The most technically demanding aspect was managing the complex state synchronisation for the Staff Scheduler. The initial design implemented had an auto-scroll to the current day, which triggered infinite rendering loops due to conflicts with the SectionList lifecycle. I addressed this by architecting a stable Monthly View that fetches data based on specific date ranges, significantly improving app stability. Additionally, on the backend, implementing Atomic Transactions for the POS was critical. Mastering MongoDB sessions to ensure that a failed inventory update would instantly roll back a sale was essential for maintaining data integrity and preventing "ghost stock."

Future Improvements:

- **Net Profit Estimation:** Currently, the app calculates Gross Profit (Revenue minus Food Cost). The next iteration will include **Labor & Utility Tracking**. By allowing managers to input fixed monthly costs (Rent, Utilities) and calculating daily labor costs from the Scheduler ($\text{Hours} \times \text{Hourly Rate}$), the app could provide a highly accurate **Net Profit** estimate for the month.
- **Offline Mode:** Implementing a local database (like SQLite) to allow waiters to take orders and view the menu even during internet outages, syncing data once the connection is restored.

AI Usage in Development

Throughout the development of CompanyManMobile, Artificial Intelligence tools were utilised as supportive instruments for brainstorming, app architecture design, and technical planning. These tools assisted in structuring the project and solving complex technical challenges. However, all AI-generated suggestions were critically reviewed, validated, and refined by the developer to ensure the final work remains original and strictly adheres to academic standards and integrity guidelines.